

Corrección de bugs en el sistema de undo/redo de PNESupport

OpenMarkov — org.openmarkov.core

Abril 2026

Índice

1. Introducción	2
2. Arquitectura del sistema de undo/redo	2
2.1. Clases clave	3
2.2. Flujo normal de una edición compuesta	4
3. Los tres bugs	5
3.1. Bug 1: Notificaciones duplicadas en redo	5
3.1.1. Síntoma	5
3.1.2. Causa	5
3.1.3. Código con el bug	6
3.2. Bug 2: Evento asimétrico en redo	6
3.2.1. Síntoma	6
3.2.2. Causa	7
3.3. Bug 3: Orden incorrecto de sub-edits en compound	7
3.3.1. Síntoma	7
3.3.2. Causa	7
3.3.3. Por qué no se manifestaba siempre	8
4. Corrección aplicada	8
4.1. Cambio 1: Eliminar <code>flattenEdit()</code> (Bugs 1 y 2)	8
4.2. Cambio 2: Invertir el orden de <code>getDoneEdits()</code> (Bug 3)	9
4.3. Actualización de llamadores	10
5. Diagramas de secuencia: antes y después	10
6. Ejemplo completo	12
6.1. Comportamiento antes de la corrección	12
6.2. Comportamiento después de la corrección	13
7. Tests de regresión	13
8. Resumen de ficheros modificados	14

1. Introducción

El sistema de undo/redo de OpenMarkov permite deshacer y rehacer cualquier modificación sobre una red probabilística (**ProbNet**). Toda modificación se encapsula en un objeto **PNEdit**, y el componente central que gestiona el historial y las notificaciones a los observadores es **PNESupport**.

Cuando el usuario ejecuta una acción compuesta en la GUI —por ejemplo, abrir el diálogo de propiedades de un nodo, modificar varias propiedades y pulsar Aceptar— la GUI agrupa los edits individuales entre llamadas a `openNewSubEditHistory()` y `closeSubEditHistory()`. Al cerrar, los edits se empaquetan en un `ListPNEdit` (un tipo de `CompoundPNEdit`) que se trata como una unidad atómica de undo: un solo `Ctrl+Z` deshace todo el grupo.

Este mecanismo contenía **tres bugs** que afectaban tanto a las notificaciones que reciben los listeners como al orden de ejecución de los sub-edits. Los bugs estaban marcados con la anotación `@ToCheck` en el código fuente, y llevaban tiempo sin resolverse. Este documento describe en detalle los tres bugs, la causa raíz común, la corrección aplicada y las pruebas que la validan.

2. Arquitectura del sistema de undo/redo

Antes de describir los bugs, es necesario entender la arquitectura del sistema. La Figura 1 muestra las clases implicadas y sus relaciones.

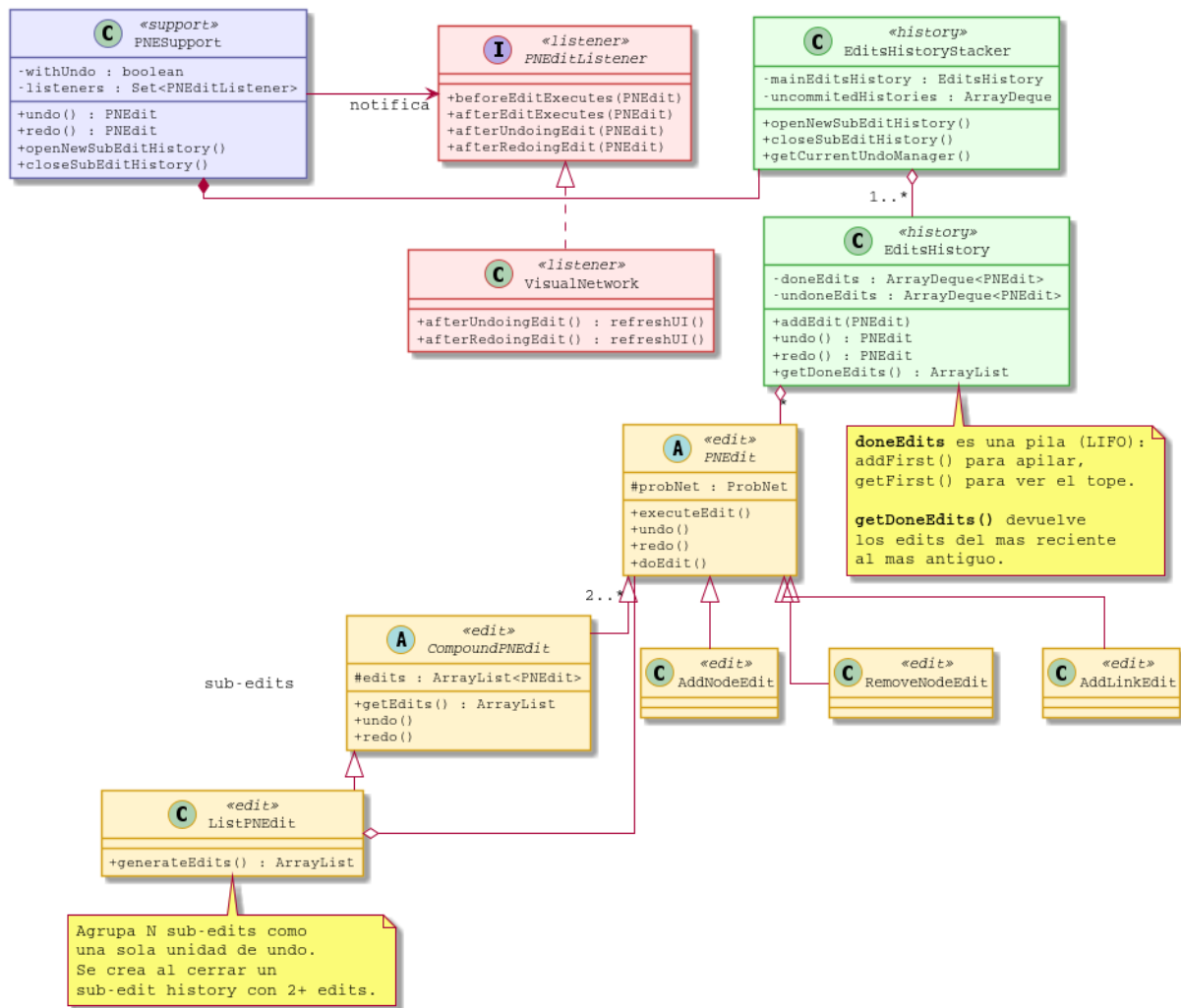


Figura 1: Clases principales del sistema de undo/redo.

2.1. Clases clave

PNESupport

Punto de entrada para undo/redo. Mantiene el conjunto de listeners (PNEditListener) y los métodos `undo()` y `redo()`, que delegan en el historial y notifican a los listeners.

EditsHistoryStacker

Gestiona una pila de historiales. Permite abrir sub-historiales anidados (transacciones) con `openNewSubEditHistory()` y cerrarlos con `closeSubEditHistory()`. Al cerrar un sub-historial con dos o más edits, los empaqueta en un `ListPNEdit`.

EditsHistory

Un historial individual con dos pilas: `doneEdits` (edits ejecutados) y `undoneEdits` (edits deshechos). Ambas son `ArrayDeque` usadas como pilas LIFO: los edits se apilan con `addFirst()` y se consultan con `getFirst()`.

CompoundPNEdit

Edit abstracto que contiene una lista de sub-edits. Su `redo()` ejecuta los sub-edits en orden directo; su `undo()` los ejecuta en orden inverso.

ListPNEdit

Implementación concreta de `CompoundPNEdit` que simplemente almacena la lista de sub-edits que recibe en el constructor.

PNEditListener

Interfaz de observador con cuatro métodos: `beforeEditExecutes`, `afterEditExecutes`, `afterUndoingEdit` y `afterRedoingEdit`.

2.2. Flujo normal de una edición compuesta

Para entender los bugs, es útil seguir el flujo completo de una edición compuesta. Supongamos que el usuario ejecuta una acción que crea dos nodos (A y B) y un arco entre ellos ($A \rightarrow B$):

1. La GUI llama a `openNewSubEditHistory()`. Esto crea un nuevo `EditsHistory` vacío en la pila.
2. Se ejecutan los edits individuales:
 - `AddNodeEdit(A).executeEdit()` — se añade a la pila: `doneEdits = [A]`
 - `AddNodeEdit(B).executeEdit()` — se apila encima: `doneEdits = [B, A]`
 - `AddLinkEdit(A  $\rightarrow$  B).executeEdit()` — se apila encima: `doneEdits = [Link, B, A]`

Cada uno notifica a los listeners con `afterEditExecutes`.

3. La GUI llama a `pneSupport.closeSubEditHistory()`. `EditsHistoryStacker` obtiene los `doneEdits` del sub-historial y los empaqueta en un `ListPNEdit`.
4. El `ListPNEdit` se añade al historial padre como un solo edit.

El punto crítico es el paso 3: `getDoneEdits()` devuelve los edits en **orden de pila** (el más reciente primero), es decir `[Link, B, A]`. Este es el orden en que se almacenan dentro del `ListPNEdit`.

La Figura 2 muestra paso a paso cómo evoluciona la pila `doneEdits` durante la ejecución de los tres edits, y cómo la lectura de la pila produce una lista en orden inverso al de ejecución — la causa raíz del Bug 3.

Evolución de la pila doneEdits y lectura al cerrar

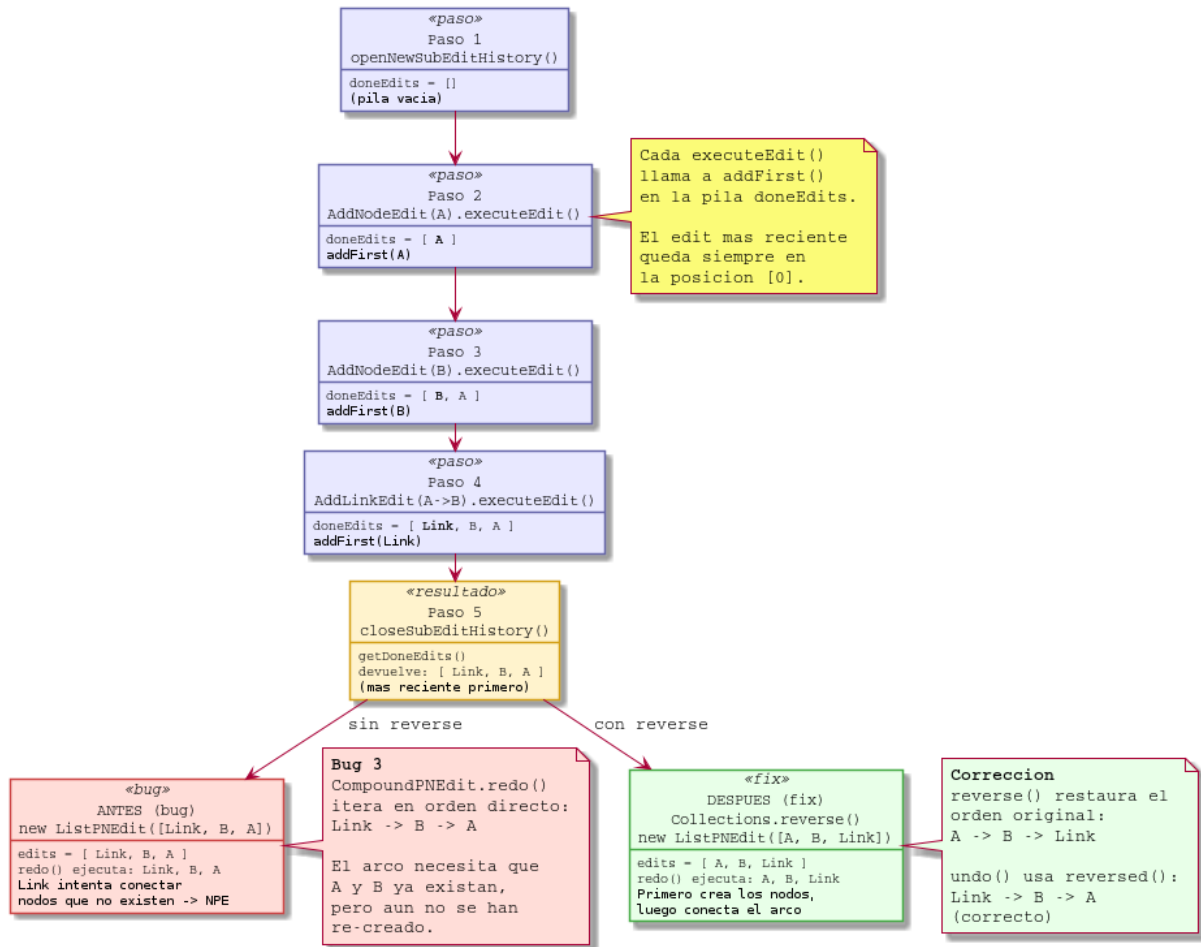


Figura 2: Evolución de la pila `doneEdits` durante la ejecución. Cada `executeEdit()` apila el edit con `addFirst()`. Al cerrar el sub-historial, `getDoneEdits()` devuelve la lista en orden de pila (más reciente primero). Sin el `reverse()`, el `ListPNEdit` almacena los edits en orden invertido.

3. Los tres bugs

3.1. Bug 1: Notificaciones duplicadas en redo

3.1.1. Síntoma

Al rehacer un compound edit, los listeners recibían **tres** notificaciones `afterRedoingEdit` en vez de una: una con el `ListPNEdit` y una con cada sub-edit.

3.1.2. Causa

El método `PNESupport.redo()` utilizaba un método auxiliar `flattenEdit()` que expandía recursivamente el compound edit en una lista plana mediante BFS (búsqueda en anchura). Dado un `ListPNEdit` con sub-edits `[B, A]`, `flattenEdit()` devolvía:

[ListPNEdit, B, A]

Y luego iteraba esta lista, notificando a cada listener con cada elemento. El resultado era que el listener recibía la notificación para el compound *y también* para cada sub-edit individual.

3.1.3. Código con el bug

```
1 public ArrayList<PNEdit> redo() {
2     var redoneEdit = editsHistoryStacker
3         .getCurrentUndoManager().redo();
4     // flattenEdit() devuelve [compound, subA, subB]
5     ArrayList<PNEdit> redoneEdits = flattenEdit(redoneEdit);
6     for (PNEdit subRedoneEdit : redoneEdits) {
7         for (PNEditListener listener : listeners) {
8             listener.afterRedoingEdit(subRedoneEdit);
9         }
10    }
11    return redoneEdits;
12 }
```

Listado 1: Método redo() con el bug (simplificado).

```
1 private ArrayList<PNEdit> flattenEdit(PNEdit edit) {
2     var editsToVisit = new ArrayDeque<PNEdit>();
3     editsToVisit.addFirst(edit);
4     var flattenedEdits = new ArrayList<PNEdit>();
5     while (!editsToVisit.isEmpty()) {
6         var e = editsToVisit.removeFirst();
7         flattenedEdits.add(e);
8         if (e instanceof CompoundPNEdit compound) {
9             compound.getEdits()
10                .forEach(editsToVisit::addLast);
11        }
12    }
13    return flattenedEdits;
14 }
```

Listado 2: Método flattenEdit() — BFS que causa la duplicación.

3.2. Bug 2: Evento asimétrico en redo

3.2.1. Síntoma

Durante la ejecución original, los listeners recibían `afterEditExecutes(compound)`. Pero en un redo, debido a `flattenEdit()`, recibían `afterRedoingEdit(subEdit)` para cada sub-edit individual. Esto creaba una **asimetría**: el listener tenía que manejar dos tipos de evento distintos para la misma operación lógica.

3.2.2. Causa

La misma que el Bug 1: `flattenEdit()` descomponía el compound en sus partes, y cada parte se notificaba con `afterRedoingEdit` en vez de notificar una sola vez con el edit completo.

3.3. Bug 3: Orden incorrecto de sub-edits en compound

3.3.1. Síntoma

Al rehacer un compound edit que contenía edits dependientes (por ejemplo, crear un nodo y luego crear un arco hacia ese nodo), el redo fallaba con `NullPointerException` porque intentaba rehacer el arco antes de que el nodo existiera.

3.3.2. Causa

La pila `doneEdits` de `EditsHistory` almacena los edits con `addFirst()`, es decir, el más reciente queda primero. El método `getDoneEdits()` simplemente copia esta pila en un `ArrayList`, devolviendo los edits en **orden inverso** al de ejecución original.

Este `ArrayList` se pasaba directamente al constructor de `ListPNEdit`, que lo almacenaba tal cual. El resultado:

- El usuario ejecuta: A, B, `Link(A→B)`
- `getDoneEdits()` devuelve: `[Link, B, A]`
- `ListPNEdit` almacena: `[Link, B, A]`

`CompoundPNEdit` ejecuta `redo()` en orden directo sobre esta lista, y `undo()` en orden inverso:

```
1  @Override public void redo() {
2      this.getEdits().forEach(PNEdit::redo);
3  }
4
5  @Override public void undo() {
6      this.getEdits().reversed().forEach(PNEdit::undo);
7  }
```

Listado 3: Métodos `redo()` y `undo()` de `CompoundPNEdit`.

Con la lista `[Link, B, A]`:

- `redo()` ejecuta: `Link.redo()`, `B.redo()`, `A.redo()` — **incorrecto**: el arco se rehace antes que los nodos. Si el arco depende de los nodos, se produce un `NullPointerException`.
- `undo()` ejecuta `reversed()`: `A.undo()`, `B.undo()`, `Link.undo()` — **incorrecto**: los nodos se deshacen antes que el arco.

3.3.3. Por qué no se manifestaba siempre

Con edits independientes (por ejemplo, dos `AddNodeEdit` sin arcos entre ellos), el orden de ejecución no importa y el undo/redo funciona correctamente. El bug solo se manifestaba con edits dependientes, como crear un arco entre dos nodos recién creados.

La Figura 3 ilustra el problema del orden con un ejemplo concreto.

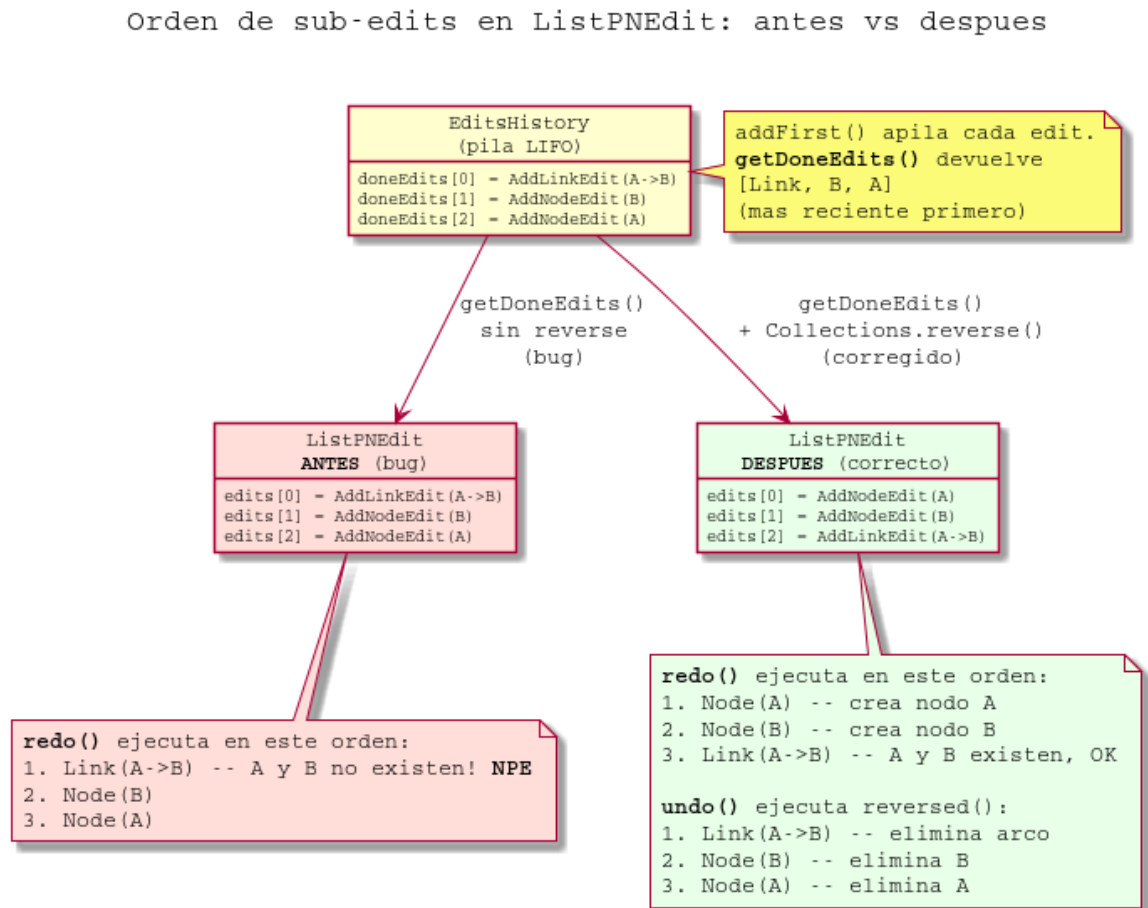


Figura 3: Orden de sub-edits dentro de `ListPNEdit`: antes (bug) y después (correcto).

4. Corrección aplicada

La corrección consta de dos cambios independientes, cada uno dirigido a una causa raíz distinta.

4.1. Cambio 1: Eliminar `flattenEdit()` (Bugs 1 y 2)

Se eliminó el método `flattenEdit()` y se simplificaron `undo()` y `redo()` para notificar una sola vez con el edit que devuelve `EditsHistory`:

```
1 public @Nullable PNEdit redo() {
2     var redoneEdit = editsHistoryStacker
```

```

3         .getCurrentUndoManager().redo();
4     if (redoneEdit != null) {
5         for (PNEditListener listener : listeners) {
6             listener.afterRedoingEdit(redoneEdit);
7         }
8     }
9     return redoneEdit;
10 }

```

Listado 4: Método redo() corregido.

```

1 public @Nullable PNEdit undo() {
2     var undoneEdit = editsHistoryStacker
3         .getCurrentUndoManager().undo();
4     if (undoneEdit != null) {
5         for (PNEditListener listener : listeners) {
6             listener.afterUndoingEdit(undoneEdit);
7         }
8     }
9     return undoneEdit;
10 }

```

Listado 5: Método undo() corregido.

Consecuencias:

- Para un edit simple, el comportamiento no cambia: el listener recibe una notificación con el edit.
- Para un compound edit, el listener recibe una sola notificación con el ListPNEdit. Si necesita los sub-edits, puede hacer instanceof CompoundPNEdit y llamar a getEdits().
- Se eliminaron las anotaciones @ToCheck de ambos métodos.
- El tipo de retorno cambió de ArrayList<PNEdit> a @Nullable PNEdit. Se actualizaron los dos llamadores que usaban el valor de retorno (ver Sección 4.3).

4.2. Cambio 2: Invertir el orden de getDoneEdits() (Bug 3)

En el método closeSubEditHistory() de la clase EditsHistoryStacker, se añadió una llamada a Collections.reverse() después de obtener los doneEdits, para restaurar el orden original de ejecución antes de pasarlo al constructor de ListPNEdit:

```

1 var lastUsedStack = this.uncommittedHistories.removeLast();
2 // getDoneEdits() devuelve el mas reciente primero (pila);
3 // invertimos para restaurar el orden original de ejecución.
4 ArrayList<PNEdit> doneEdits = lastUsedStack.getDoneEdits();
5 Collections.reverse(doneEdits);
6 PNEdit stackAsEdit = switch (doneEdits.size()) {
7     case 0 -> null;
8     case 1 -> doneEdits.getFirst();
9     default -> new ListPNEdit(

```

```

10     doneEdits.getFirst().getProbNet(), doneEdits);
11 };

```

Listado 6: `closeSubEditHistory()` corregido (fragmento).

Con esta corrección, si el usuario ejecutó A, B, `Link(A→B)`:

- `getDoneEdits()` devuelve: [Link, B, A]
- Tras `reverse()`: [A, B, Link]
- `ListPNEdit` almacena: [A, B, Link]
- `redo()` ejecuta: A, B, Link — **correcto**
- `undo()` ejecuta `reversed()`: Link, B, A — **correcto**

4.3. Actualización de llamadores

El cambio de tipo de retorno de `undo()` y `redo()` (de `ArrayList<PNEdit>` a `@Nullable PNEdit`) requirió actualizar dos llamadores que usaban el valor de retorno:

Tabla 1: Llamadores actualizados por el cambio de tipo de retorno.

Clase	Cambio
<code>EditsHistoryPlugin(org.openmarkov.full)</code>	Usaba <code>.undo().contains(edit)</code> en un bucle <code>while</code> . Cambiado a un bucle <code>do-while</code> que compara el edit devuelto con <code>equals()</code> .
<code>EditorInputHandler(org.openmarkov.gui)</code>	Usaba <code>.undo().stream().anyMatch(...)</code> . Cambiado a un bucle <code>do-while</code> que comprueba <code>instanceof AddNodeEdit</code> .

Todos los demás llamadores (más de 20 en GUI, tests y learning) usaban `undo()/redo()` sin capturar el valor de retorno, por lo que no requirieron cambios.

5. Diagramas de secuencia: antes y después

Las Figuras 4 y 5 muestran el flujo de ejecución de `redo()` con un compound edit, antes y después de la corrección.

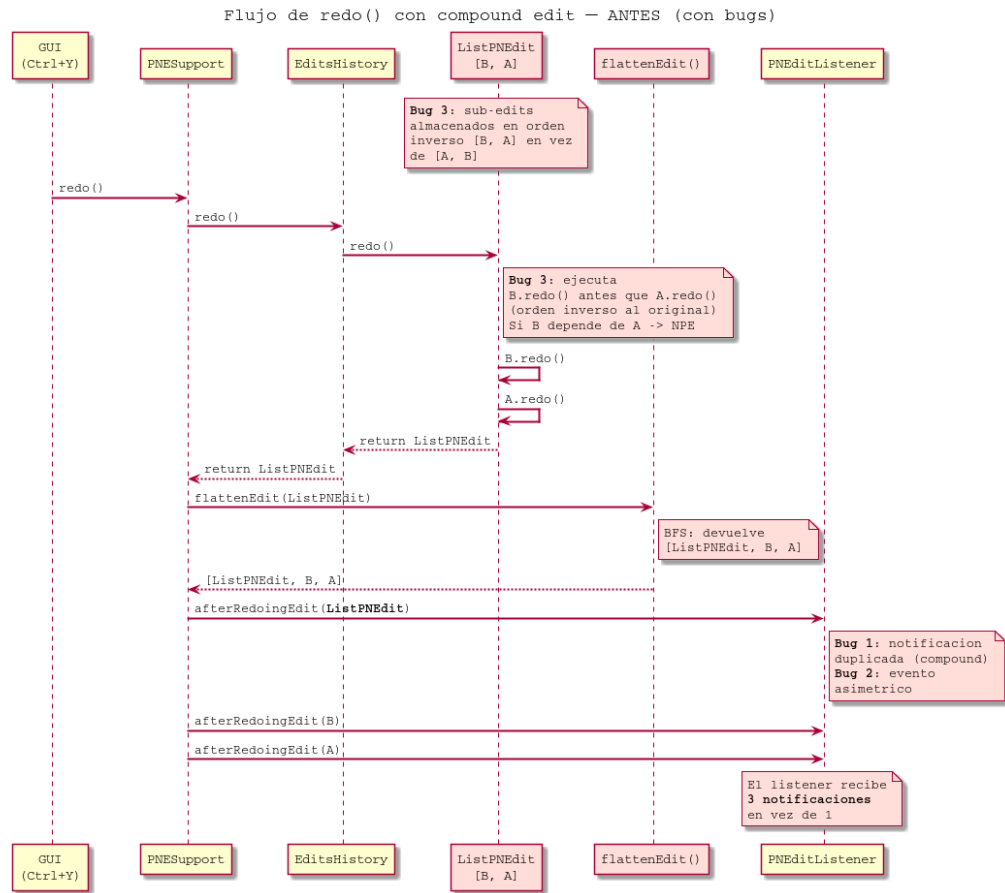


Figura 4: Flujo de redo() **antes** de la corrección. Se observan los tres bugs: orden incorrecto (Bug 3), notificaciones duplicadas (Bug 1) y evento asimétrico (Bug 2).

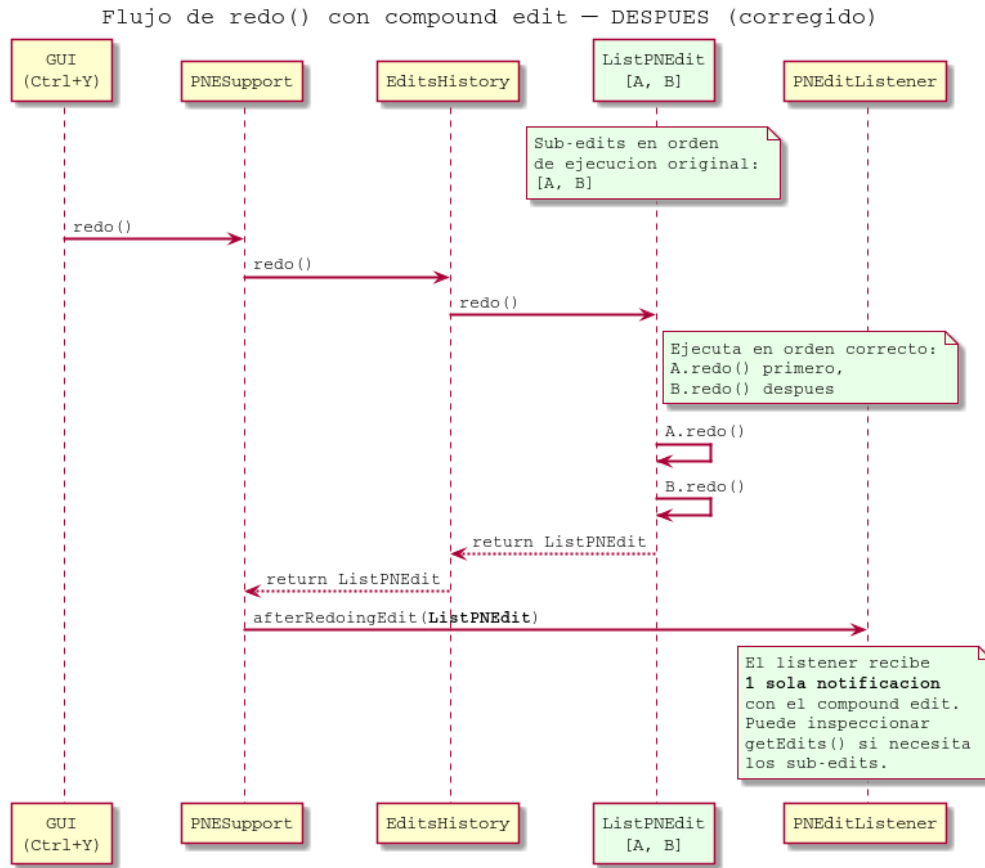


Figura 5: Flujo de redo() **después** de la corrección. Los sub-edits se ejecutan en orden correcto y el listener recibe una única notificación.

6. Ejemplo completo

Para ilustrar el impacto práctico, consideremos el siguiente escenario real en la GUI:

1. El usuario abre el diálogo de propiedades de un nodo B.
2. Añade un padre A (se ejecuta AddNodeEdit(A) y AddLinkEdit(A→B)).
3. Pulsa Aceptar (se cierra el sub-historial).
4. Pulsa Ctrl+Z (undo): se deshace todo correctamente.
5. Pulsa Ctrl+Y (redo).

6.1. Comportamiento antes de la corrección

En el paso 5, redo() ejecutaba los sub-edits del ListPNEdit en orden de pila: primero AddLinkEdit(A→B), luego AddNodeEdit(A). El arco intentaba conectar dos nodos que aún no existían, produciendo:

```

NullPointerException: Cannot invoke
    "Node.setPotentials(List)"
  
```

```
because "this.nodeTo" is null
```

6.2. Comportamiento después de la corrección

En el paso 5, `redo()` ejecuta los sub-edits en orden original: primero `AddNodeEdit(A)`, luego `AddLinkEdit(A→B)`. El nodo A existe cuando se crea el arco. El redo completa sin error y la red queda en el mismo estado que tras el paso 2.

7. Tests de regresión

Se creó la clase `PNESupportPropertyTest` con 7 tests usando `jqwik` (`@Property` y `@Example`):

Tabla 2: Tests de regresión para el sistema de undo/redondo.

Test	Qué verifica
<code>simpleEdit_undoNotifiesListenerExactlyOnce</code>	Undo de un edit simple produce exactamente una notificación.
<code>simpleEdit_redoNotifiesListenerExactlyOnce</code>	Redo de un edit simple produce exactamente una notificación.
<code>simpleEdit_undoRedoPreservesNetworkState</code>	Tras undo + redo de un edit simple, el estado de la red es idéntico al original.
<code>compoundEdit_undoRedoNotifiesExactlyOnce</code>	Undo y redo de un compound edit producen exactamente una notificación cada uno, y la notificación lleva el <code>CompoundPNEdit</code> . Regresión para Bugs 1 y 2.
<code>compoundEdit_subEditsInOriginalOrder</code>	Los sub-edits dentro del <code>ListPNEdit</code> están en el orden original de ejecución. Regresión para Bug 3.
<code>compoundEdit_redoDependentEditsDoesNotThrow</code>	Redo de un compound con edits dependientes (nodos + arco) no lanza excepción. Regresión para Bug 3.
<code>compoundEdit_undoRedoPreservesNetworkState</code>	Tras undo + redo de un compound con edits dependientes, la red conserva todos los nodos y arcos.

Todos los tests pasan con la corrección aplicada y fallan sin ella, lo que confirma tanto la existencia de los bugs como la corrección.

8. Resumen de ficheros modificados

Tabla 3: Ficheros modificados en la corrección.

Módulo	Fichero	Cambio
core	<code>PNESupport.java</code>	Eliminados <code>flattenEdit()</code> y <code>@ToCheck</code> . Tipo de retorno de <code>undo()/redo()</code> cambiado a <code>Nullable PEdit</code> . Notificación única.
core	<code>EditsHistoryStacker.java</code>	Añadido <code>Collections.reverse(doneEdits)</code> en <code>closeSubEditHistory()</code> .
full	<code>EditsHistoryPlugin.java</code>	Adaptado al nuevo tipo de retorno.
gui	<code>EditorInputHandler.java</code>	Adaptado al nuevo tipo de retorno.
core (test)	<code>PNESupportPropertyTest.java</code>	7 tests de regresión (nuevo fichero).

9. Impacto en la GUI

El principal listener de undo/redo en la GUI es `VisualNetwork`, que implementa los métodos de notificación de undo y redo con una llamada a `refreshUI()` que repinta toda la vista. Este listener es tolerante a notificaciones duplicadas o en orden incorrecto, ya que siempre repinta completamente. Por esta razón, los bugs no causaban problemas visibles en la mayoría de los casos de uso.

Sin embargo, otros listeners (`ValuesTable`, `ICIValuesTable`, constraints como `NoCycle`) podrían haberse visto afectados por notificaciones duplicadas o en orden incorrecto. Además, el Bug 3 causaba un `NullPointerException` irrecuperable al rehacer compound edits con dependencias, lo que sí era un fallo visible para el usuario.